

Password Strength Analyzer with Custom Wordlist Generator

1. Abstract:

This project focuses on developing a Python-based tool that evaluates the strength of user passwords using the zxcvbn library and generates a custom wordlist based on user-provided information. The tool demonstrates fundamental cybersecurity concepts, including password entropy, weakness detection, and dictionary-based attack preparation. By combining analysis and wordlist generation, the project provides practical learning in both defensive and offensive security techniques.

2. Introduction:

Weak passwords remain one of the most common security vulnerabilities in digital systems. Understanding password strength and how attackers build wordlists is an essential skill in cybersecurity. This project implements a simple but effective password analysis tool that not only rates password strength but also generates personalized wordlists that mimic attacker methods. The program accepts user data (name, date, pet name, etc.), applies transformations such as leetspeak and year variations, and saves the generated list to a text file for practical use in security learning environments.

3. Tools Used:

Python 3.x – Core programming language.
zxcvbn – Password strength evaluation library.
NLTK – Optional for basic word processing.
Tkinter / CLI – Interface used for interacting with the tool.
File Handling (txt export) – Saves generated wordlists.
VS Code / Terminal – Development and testing.
Git & GitHub – Version control and project deployment.

4. Steps Involved in Building the Project:

Step 1: Planning the Application

Identified required features: password analysis, user input collection, wordlist generation, and file export. Selected libraries (zxcvbn, nltk, tkinter/CLI).

Step 2: Setting Up the Environment

Installed required libraries using: `pip install zxcvbn nltk`
Verified Python and library versions.

Step 3: Developing the Password Strength Checker

Imported zxcvbn and processed user-entered passwords.
Extracted score, estimated crack time, and feedback. Displayed results in GUI or terminal.

Step 4: Designing the Wordlist Generator

Collected user inputs (name, DOB, pet name, favorite word, etc.). Stored all unique entries in a list.

Step 5: Exporting Wordlist

Saved generated words into a .txt file using Python's file handling.

Step 6: Building the Interface (Optional GUI with Tkinter) Created simple input fields and buttons.

Step 7: Testing the Application

Checked password evaluation accuracy.

Step 8: Finalizing and Uploading to GitHub

Added README file, project description, and screenshots.

5. Conclusion:

The Password Strength Analyzer and Wordlist Generator successfully demonstrates real-world cybersecurity concepts, including password robustness, entropy, and dictionary attack modeling. The tool provides hands-on understanding of how attackers build wordlists and how users can create stronger passwords. This project strengthened skills in Python programming, file handling, GUI development, and the use of external libraries, while also enhancing practical cybersecurity awareness.

Internship: Elevate Labs,

Prepared by: P. SUSMITHA,

Date: 25/11/2025.